

Interview Preparation Guide

Based on the Resume of Prashant Singh — Software Engineer

Candidate: Prashant Singh | New Delhi, India

Target role: Software Engineer (Full-Stack / MERN / Next.js)

Education: B.Tech CSE, Jaypee University of Engineering & Technology (2023–2027)

Current role: Web Developer, GoDigital Agency

Every resume line → likely question → model answer → key terms explained



How to use this guide: Each section mirrors a section of your resume. For every bullet point you can expect interviewers to ask a mix of (a) "walk me through this" questions, (b) technical deep-dives on the tools you name, and (c) "why/how" follow-ups that test whether you actually did the work. Model answers are written in first person so you can adapt them — replace them with your own specifics and numbers where you can. A glossary at the end defines every technical term on your resume in plain language.

1 Summary / Profile Questions

Q Walk me through your resume / Tell me about yourself.

A: I'm a full-stack developer currently pursuing my B.Tech in Computer Science at Jaypee University, graduating in 2027. I specialize in the MERN stack and Next.js. Right now I work as a Web Developer at GoDigital Agency, where I ship production client websites using Next.js 16 and React 19 with 3D/motion UI. Before that, I interned at BJjewelry.in building a full-stack B2B jewellery platform. Outside of work I've built a real-time chat app with Socket.io and published my own npm component library, virtual-ui-lib. I enjoy writing clean, maintainable code and shipping features on schedule end-to-end — from the database to the UI.

Q What does "full-stack developer specializing in MERN and Next.js" actually mean to you?

A: MERN stands for MongoDB, Express.js, React, and Node.js — a JavaScript-only stack that lets me use one language across the browser and the server. Being "full-stack" means I'm comfortable owning a feature end-to-end: designing a MongoDB schema, writing an Express/Node API, and building the React or Next.js UI that consumes it. Next.js sits on top of React and adds server-side rendering, routing, and API routes, which is what most of my recent production work is built on.

Q You say you focus on "clean, maintainable code that ships on schedule" — how do you balance code quality against deadlines?

A: I try to keep components small and single-purpose, name things clearly, and avoid premature abstraction — I'd rather duplicate a few lines than build a generic system for a case that only exists once. When a deadline is tight I'll ship the simplest version that's still correct and typed properly (I lean on TypeScript to catch mistakes early), and note down what I'd refactor later rather than gold-plating a feature no one asked for yet.

Q What's the difference between a website you'd call a "production" site versus a demo/prototype?

A: A production site is live, used by real users or clients, and has to handle real constraints — deployment pipelines, custom domains, DNS, performance, error handling, and content that non-developers can update (which is why I use a CMS like Sanity). A prototype just needs to demonstrate an idea; it doesn't need to be resilient, SEO-friendly, or maintainable by someone else.

2 Education

Q You're still in college (2023–2027) but already working — how do you manage both?

A: I treat my coursework and my professional work as complementary — CS fundamentals like data structures, DBMS, and networking give me the theory, and the agency/internship work is where I apply it under real deadlines and real user feedback. I manage time by working in focused blocks and prioritizing whichever has the nearer deadline; the practical experience has actually made some of my coursework easier to reason about (e.g., understanding indexes after using MongoDB/PostgreSQL in production).

Q Which CS fundamentals (DSA, OS, DBMS, Networks) are you strongest in, and how have you applied them in your projects?

A: DBMS is where I've applied the most — designing MongoDB document schemas for the chat app and jewellery platform, and comparing that against relational modeling in PostgreSQL/SQL. Networking concepts (client-server, HTTP, WebSockets) came up directly when building Socket.io real-time messaging and JWT-based auth over HTTPS. I'd want to keep sharpening DSA through practice (I have a LeetCode profile) since it's the most commonly tested in interviews even though it's less visible day-to-day in web development.

3 Experience — Web Developer, GoDigital Agency (Jun 2026–Present)

Shipped production client websites (GoDigital Agency site, Jiya JR e-commerce) using Next.js 16, React 19, TypeScript, with 3D/motion UI (React Three Fiber, Spline, GSAP)

Q Walk me through how you'd build and ship a client website at an agency, start to finish.

A: It starts with the client brief and any Figma designs, then I scaffold the project in Next.js with the App Router, set up the page structure and shared layout, and build components in React with TypeScript for type safety. For content that changes often I wire it to Sanity CMS instead of hardcoding it. Once the core pages work, I add motion/3D polish, test responsiveness and performance, then push to Vercel, connect the client's custom domain via DNS, and monitor after launch.

Q Why Next.js instead of plain React for client-facing sites?

A: Client sites usually need good SEO and fast first paint, which plain client-side React (a single-page app) doesn't give you out of the box. Next.js adds server-side rendering and static generation so pages are pre-rendered with real content for search engines and load faster for users. It also gives me file-based routing and built-in API routes, so I don't need a separate backend for simple things like a contact form handler.

Q What's new/different in React 19 and Next.js 16 compared to earlier versions?

A: React 19 matured Server Components and Actions (form submissions that call server functions directly without hand-rolled API glue), better built-in handling of async data and refs, and improved hydration. Next.js 16 builds on the App Router with Server Components as the default, more granular caching controls, and faster builds/dev server via Turbopack. Practically, it means less boilerplate for data fetching and mutations compared to the old pages-router + `getServerSideProps` model.

Q Why TypeScript over plain JavaScript for these projects?

A: TypeScript catches whole classes of bugs — wrong prop types, undefined values, typos in field names — at compile time instead of in production. On a client site, that matters because a runtime crash is visible and costly. It also makes the codebase easier for a teammate (or future me) to understand, since types document what a function expects and returns without needing extra comments.

Q What is React Three Fiber and why use it instead of plain Three.js?

A: React Three Fiber is a React renderer for Three.js — it lets me describe a 3D scene declaratively as React components (`<mesh>`, `<light>`, etc.) instead of imperatively creating and managing Three.js objects by hand. That means the 3D scene participates in React's state and lifecycle like everything else on the page, so it's easier to keep in sync with the rest of the UI and to reuse pieces as components.

Q What role does Spline play versus React Three Fiber, and how does GSAP fit in?

A: Spline is a visual 3D design tool — designers or I can build and export a 3D scene/model without hand-coding the geometry, then embed it in the site. React Three Fiber is for when I need custom, code-driven 3D that reacts to app state. GSAP (GreenSock Animation Platform) is a JavaScript animation library I use for timeline-based motion — scroll-triggered reveals, sequenced transitions — that's smoother and more controllable than raw CSS transitions for complex sequences.

Built headless Sanity CMS backends with custom schemas and Portable Text

Q What does "headless CMS" mean, and why did the client need one?

A: A headless CMS stores and manages content but has no built-in front-end — it just exposes the content through an API, and I build the presentation layer myself in Next.js. That's the opposite of something like WordPress, where the CMS and the rendered site are bundled together. Clients need it so their marketing team can edit text, images, and page content themselves without touching code or waiting on a developer for every change.

Q What's a "custom schema" in Sanity, and what's Portable Text?

A: A schema in Sanity is a JavaScript/TypeScript definition of a content type's shape — e.g., a "Blog Post" schema might define a title string, a slug, a hero image, and a body field. Portable Text is Sanity's rich-text format: instead of storing HTML, it stores structured JSON blocks (paragraphs, headings, marks like bold/links) that I render into React components on the front end. That structure is safer than raw HTML (no injection risk) and more flexible than a plain string, since I control exactly how each block type renders.

Configured Vercel deployment (custom domains, DNS); integrated Zustand, Framer Motion, Resend

Q What's involved in pointing a client's custom domain at a Vercel deployment?

A: I add the domain in the Vercel project settings, then update the client's DNS records — typically an A record or ALIAS pointing to Vercel's IP, plus a CNAME for the www subdomain — through their domain registrar. Vercel then auto-provisions an SSL certificate for HTTPS once the DNS propagates. I also set up preview deployments per branch/PR so the client can review changes before they go live on the main domain.

Q Why Zustand instead of Redux or React Context for state management?

A: Zustand is a much smaller, less boilerplate-heavy state library — no providers, actions, or reducers required, just a hook-based store. For the scale of state these client sites need (cart state, UI toggles, form state) it's simpler than Redux's ceremony, and unlike React Context it doesn't re-render the whole subtree on every change — components only re-render when the specific piece of state they read changes.

Q What is Framer Motion used for, and what is Resend?

A: Framer Motion is a React animation library — I use it for declarative UI animations like page transitions, fade/slide-ins, and gesture-based interactions (hover, drag), which is simpler than hand-writing CSS keyframes for React component state changes. Resend is a transactional email API for developers — I use it to send emails triggered by app events, like a contact-form confirmation or an order notification, from a Next.js API route or Server Action.

Q How do you decide which of these libraries to reach for on a new client project?

A: I start from what the page actually needs. If there's shared state across components (cart, auth, theme), I reach for Zustand rather than prop-drilling. If a design calls for motion, Framer Motion covers most UI-level animation, and I only bring in GSAP for more complex, timeline-driven sequences. 3D only goes in if the design specifically calls for it — I don't add it for its own sake since it adds bundle size and complexity.

4 Experience — Full Stack Developer Intern, BJJewelry.in (Jan–Mar 2026)

Built a full-stack B2B jewellery platform (100+ products) with Next.js and Supabase; improved page load performance by 30%

Q What makes a platform "B2B" versus a normal e-commerce site, and how did that shape your build?

A: B2B (business-to-business) means the buyers are other businesses, not individual consumers — so the platform needed things like bulk quantities, wholesale-style pricing/catalog browsing, and often a more information-dense product view (specs, certifications) rather than a purely visual consumer storefront. It shaped decisions like prioritizing fast catalog browsing and export/reporting tools over things like a flashy checkout animation.

Q What is Supabase, and why choose it over building your own backend or using MongoDB?

A: Supabase is a backend-as-a-service built on PostgreSQL — it gives me a hosted relational database, auto-generated REST/real-time APIs, authentication, and file storage without standing up my own server. I chose it here because the product catalog data (100+ products with structured attributes, categories, relationships) fit a relational model well, and Supabase let me move fast without hand-writing a full Express/Node backend for standard CRUD operations.

Q You mention a 30% page load improvement — walk me through how you'd actually measure and achieve that.

A: I'd start by profiling with Chrome DevTools / Lighthouse to find what's slow — usually unoptimized images, unnecessary client-side JavaScript, or components re-rendering too much. Concretely: I used Next.js's built-in Image component for automatic image optimization and lazy loading, restructured component architecture so heavy components weren't re-rendering on unrelated state changes, and made sure data-heavy pages used server rendering so the browser wasn't waiting on client-side fetches. I'd measure before/after using Lighthouse scores or Core Web Vitals (LCP, TTFB) to get the concrete percentage.

Q What does "optimizing component architecture" mean concretely?

A: Mainly three things: splitting large components so each has a narrow reason to re-render, memoizing expensive computations or components that don't need to update every render, and pushing state as low/local as possible instead of putting everything in a top-level store that causes wide re-render cascades. It also means deciding which components can be Server Components (no client JS shipped) versus which actually need interactivity as Client Components.

| Sanity CMS admin dashboard + CSV/Excel export; UI/UX prototypes in Figma

Q How did you implement CSV/Excel export functionality?

A: I'd query the product/order data from the database, transform it into rows, and generate the file — for CSV that's straightforward string joining or a small library, and for Excel I'd use a library like SheetJS (xlsx) to build a proper .xlsx file with headers and formatting, then trigger a browser download. The goal was letting non-technical staff pull reports without needing database access.

Q What was your Figma-to-code workflow?

A: I'd design or receive the prototype in Figma with defined spacing, typography, and color tokens, then translate those into Tailwind CSS utility classes and reusable React components so the implementation stayed pixel-close to the design. Prototyping in Figma first also let me validate UX flows — like the admin dashboard's navigation — with stakeholders before writing code, which is cheaper than redoing a built feature.

Q As an intern, how did you handle situations where you didn't know how to do something?

A: I'd timebox my own research first — official docs, then targeted searches — and if I was still stuck after a reasonable effort, I'd ask a specific, well-framed question rather than a vague one, showing what I'd already tried. That kept me from blocking the team while still respecting their time.

5 Project — Realtime Chat App (MERN, Socket.io, JWT — Nov 2025)

Q Walk me through the architecture of the chat app.

A: The frontend is React 19 with Vite, using Zustand for state (current user, messages, online users) and React Router for navigation between login/register and the chat window. The backend is Express with MongoDB storing users and messages. When a user connects, a Socket.io connection is established alongside the normal REST API, and the server tracks connected sockets to broadcast online/offline status and relay messages in real time.

Q How does Socket.io actually achieve real-time messaging — what's happening under the hood?

A: Socket.io opens a persistent, bidirectional connection between client and server (WebSocket when available, falling back to HTTP long-polling if not), instead of the client repeatedly polling an HTTP endpoint. Both sides can emit named events at any time — when I send a message, the client emits a "sendMessage" event, the server receives it, saves it to MongoDB, and emits a "newMessage" event to the specific recipient's socket so it appears instantly without a page refresh.

Q How did you track "online status"?

A: I maintained a map of `userId` → `socketId` on the server, populated when a user connects (I pass their `userId` when establishing the socket connection) and removed on disconnect. I broadcast the list of currently online user IDs to all connected clients whenever it changes, so the UI can show a live indicator next to each contact.

Q Explain your authentication flow: JWT + bcrypt.

A: On signup, I hash the user's password with `bcrypt` (which salts and one-way-hashes it, so I never store plaintext passwords) before saving to MongoDB. On login, I compare the submitted password against the stored hash with `bcrypt.compare`. If it matches, I sign a JSON Web Token (JWT) containing the user's ID, set it as an `httpOnly` cookie (or return it to be sent in an Authorization header), and every protected route verifies that token to identify the requesting user without hitting the database for a session lookup.

Q Why JWT instead of traditional session-based auth?

A: JWTs are stateless — the server doesn't need to store session data anywhere (no session table/store), because the token itself is signed and carries the user's identity; the server just verifies the signature. That makes it easy to scale across multiple servers without shared session storage. The tradeoff is that a JWT can't be instantly revoked the way a server-side session can, so I keep expiry times reasonably short and store it in an `httpOnly` cookie to reduce XSS risk.

Q What is Cloudinary, and why not just store images in MongoDB or on your own server?

A: Cloudinary is a cloud media management service — I upload images to it and it handles storage, CDN delivery, and on-the-fly transformations (resizing, format conversion) via URL parameters. Storing binary image data directly in MongoDB bloats the database and hurts query performance, and self-hosting file storage means I'd have to build my own CDN, resizing, and backup story. Cloudinary just gives me a URL back that I store in the document instead of the image itself.

Q Why Vite instead of Create React App for the frontend?

A: Vite uses native ES modules during development, so the dev server starts almost instantly and hot-reloads changes very fast, regardless of project size, because it doesn't bundle the whole app up front like CRA's Webpack-based setup does. It also ships a much simpler, faster production build via Rollup. CRA is effectively unmaintained now, so Vite is the more current default for new React projects that don't need a framework like Next.js.

Q What would you change or add if you kept building this chat app?

A: I'd add message delivery/read receipts, typing indicators (which Socket.io handles well with a lightweight event), pagination/infinite scroll for message history instead of loading it all at once, and probably move from polling MongoDB for unread counts to computing them incrementally. I'd also add rate limiting on the message endpoint to prevent abuse.

6 Project — Virtual UI, React Component Library (Jan 2026)

Q What does it actually take to build and publish an npm package?

A: I structured the components in a `src` folder, set up a build step (bundling with something like Rollup/tsup to output both CommonJS and ES module builds plus TypeScript type declarations), configured `package.json` with the correct name, version, entry points (`"main"/"module"/"types"` fields), and `peerDependencies` (React, so consumers use their own React install instead of bundling a second copy). Then ``npm login`` and ``npm publish`` push it to the registry, and semantic versioning (major.minor.patch) governs how I release updates.

Q What's a `peerDependency` and why does it matter for a component library?

A: A `peerDependency` declares that my package needs React to be present in the host app, but doesn't bundle its own copy of it. If I bundled React directly, an app installing my library could end up with two React instances, which breaks hooks and context. Marking React as a peer dependency means my components use whatever React version the consuming app already has.

Q Describe some of the 15+ components you built — how did you design them to be reusable across different consumer apps?

A: Things like navbars, sidebars, cards, loaders, OTP inputs, and charts. To keep them reusable, I designed each around props for content and behavior rather than hardcoded values, kept styling configurable (so consumers aren't locked into my exact theme), and avoided baking in app-specific logic — e.g., the OTP input just exposes an `onComplete` callback with the entered code rather than assuming how the consumer verifies it.

Q Why build a documentation/playground site instead of just a README?

A: A README shows static code snippets, but a live playground lets someone see the component rendered, tweak props, and edit code in-browser to understand behavior before installing anything — much lower friction for adoption. It also doubled as a way to visually test every component myself during development.

Q What is Redux Toolkit, and how does it differ from the Zustand you used elsewhere?

A: Redux Toolkit is the modern, opinionated way to write Redux — it reduces the boilerplate of classic Redux (action types, action creators, manually written reducers, immutable update logic) via `createSlice`, which lets me write "mutating" logic that's actually made immutable under the hood using Immer. I used it for the playground site partly for practice with it and because the app's state (editor content, live preview, selected component) benefited from Redux's centralized DevTools-inspectable store. Zustand is lighter-weight for simpler global state; Redux Toolkit is more structured and has stronger tooling for larger, more complex state.

Q How did the "live, in-browser code editing" feature work?

A: The playground takes the code typed into an editor component, and instead of sending it to a build server, transforms/evaluates it client-side (e.g., via Babel's standalone transform or a sandboxed iframe) so the JSX becomes runnable JavaScript, then renders the result live as the user types. The Express/MongoDB API handled persistence — saving snippets and user accounts — separately from that live-preview logic.

Q What was the hardest part of building and shipping this library, and what would you do differently?

A: Getting the build/bundling config right was the trickiest part — making sure TypeScript types, CSS, and both module formats all worked correctly for consumers with different bundlers. If I did it again, I'd add automated visual regression tests earlier and set up CI to run the build and publish steps automatically on a tagged release, rather than doing it by hand.

7 Technical Skills — Deep-Dive Questions

Languages: C, C++, JavaScript, TypeScript

Q Why learn C/C++ if most of your work is JavaScript/TypeScript?

A: C and C++ are part of my CS coursework and they build a low-level understanding — memory management, pointers, how data structures actually work under the hood — that makes me a better JavaScript developer too, since I understand what's happening beneath abstractions like garbage collection and objects/arrays. They're also the languages I use for DSA practice.

Q What's the difference between "==" and "===" in JavaScript, and where does TypeScript's type system fit in?

A: "==" performs type coercion before comparing (so "5" == 5 is true), while "===" checks both type and value without coercion — I always use "===" to avoid subtle bugs. TypeScript adds a static type layer on top of JavaScript that's checked at compile time (via the TypeScript compiler, tsc) and erased before runtime — it doesn't change how the code runs, it just catches type mismatches before the code ever executes.

Frameworks & Libraries: React.js, Next.js, Node.js, Express.js, Tailwind CSS

Q Explain the Virtual DOM and why React uses it.

A: The Virtual DOM is an in-memory, lightweight representation of the real DOM. When state changes, React builds a new virtual DOM tree, diffs it against the previous one, and only applies the minimal set of real DOM updates needed. Direct DOM manipulation is comparatively slow, so batching and minimizing real DOM writes this way keeps UI updates fast.

Q What's the difference between Node.js and Express.js?

A: Node.js is a JavaScript runtime that lets JavaScript run outside the browser (on a server), built on Chrome's V8 engine. Express.js is a minimal web framework built on top of Node's built-in http module — it adds routing, middleware, and request/response helpers so I don't have to hand-parse URLs and headers myself for every API endpoint.

Q Why Tailwind CSS instead of plain CSS or a component library like Bootstrap?

A: Tailwind gives me utility classes (e.g., flex, px-4, text-lg) directly in the markup, so I can style components without switching files or naming CSS classes, and it avoids CSS growing unboundedly since unused utility classes are purged at build time. Compared to Bootstrap, I'm not fighting pre-built component opinions — I compose my own design system from primitives, which suited the custom, design-led client sites I built.

Databases: MongoDB, PostgreSQL, Supabase, SQL

Q When would you choose MongoDB over PostgreSQL, and vice versa?

A: MongoDB (NoSQL, document-based) fits data that's naturally nested/variable-shaped and where I need to move fast without rigid schema migrations — like chat messages or flexible content. PostgreSQL (relational/SQL) fits data with clear relationships and where I need strong consistency, joins, and constraints — like a product catalog with categories, inventory, and orders that reference each other. I used MongoDB for the chat app's message/user data and PostgreSQL (via Supabase) for the jewellery platform's structured catalog.

Q Explain a SQL JOIN in simple terms.

A: A JOIN combines rows from two tables based on a related column — for example, joining an "orders" table to a "products" table on `product_id` lets me pull the product name into an order query without duplicating product details inside every order row. It's the relational-database way of representing relationships instead of nesting everything in one document like MongoDB would.

Q What's an index, and why does it matter for performance?

A: An index is a separate data structure (usually a B-tree) the database maintains on a column so it can find matching rows without scanning the entire table. Without an index, a query on, say, a user's email has to check every row (a full table scan); with one, it can jump almost directly to the match. The tradeoff is indexes add overhead to writes and use extra storage, so I only add them on columns I actually query/filter on frequently.

Tools & Platforms: Git, GitHub, Vercel, Sanity CMS, Figma

Q Walk me through your typical Git workflow on a team project.

A: I branch off main for each feature or fix, commit in small, logical chunks with clear messages, and push to open a pull request on GitHub for review. I rebase or merge in the latest main before merging to avoid stale conflicts, and I use PR review comments as a checkpoint before anything lands on main, which is what triggers a Vercel deployment.

Q What's the difference between Git and GitHub?

A: Git is the version control tool itself — it runs locally and tracks changes, branches, and history in a repository. GitHub is a hosted platform built around Git that adds remote storage, collaboration features like pull requests and code review, issue tracking, and CI/CD integration. I could use Git without GitHub, but GitHub is what makes team collaboration and deployment (via Vercel's Git integration) practical.

8 General / Behavioral Questions (Resume-Driven)

Q Tell me about a time you had to learn a new technology quickly for a project.

A: When I joined BJewelry.in, I hadn't used Supabase before. I spent the first couple of days going through their docs and building a throwaway CRUD prototype to understand auth, row-level security, and the client SDK, before wiring it into the real product schema. Learning it hands-on with a small scoped task first, rather than reading everything up front, got me productive fast.

Q Describe a challenging bug you had to fix.

A: [Adapt to a real example.] A good structure: describe the symptom (e.g., online-status indicators in the chat app going stale after reconnects), how you isolated it (checked whether the socket's disconnect event was firing, found duplicate socket registrations for the same user across tabs), the fix (keyed the online-users map by userId with a set of socketIds instead of one-to-one), and what you'd do to prevent it next time (add a test around reconnect behavior).

Q How do you handle receiving critical feedback on your code?

A: I treat it as information about the code, not about me — I ask clarifying questions if I don't understand the concern, and I've found that feedback I initially disagreed with often pointed at a real edge case I hadn't considered. On agency work especially, client and reviewer feedback is how I calibrate what "production-ready" actually means beyond what works on my machine.

Q Where do you see yourself needing to grow as an engineer?

A: My production experience is heavily front-end/full-stack-JS focused, so I want to deepen my systems-level knowledge — things like scaling backends beyond a single server, deeper database performance tuning, and more rigorous testing practices (I'd like to write more automated tests rather than relying on manual QA). I'm also continuing to build DSA skills through LeetCode alongside the applied work.

Q Why are you interested in this role / what are you looking for next?

A: [Tailor to the specific company.] Structure: connect what you've done (shipping full production features across the stack, from DB to 3D/motion UI) to what the role needs, and name something specific about their product/engineering culture that draws you in, rather than a generic answer.

9 Glossary — All Technical Terms on the Resume, Explained

Term	Plain-Language Explanation
MERN Stack	MongoDB + Express.js + React + Node.js — a full JavaScript web development stack from database to browser.
Next.js	A React framework adding server-side rendering, file-based routing, and API routes for production-grade websites.
React.js	A JavaScript library for building UIs from reusable components, using a Virtual DOM to efficiently update the page.
React 19	Latest major React version; matured Server Components, Actions (server-callable form handlers), and simplified data/ref handling.
TypeScript	A superset of JavaScript that adds static types, checked at compile time to catch bugs before runtime.
Sanity CMS	A headless content management system — stores structured content and serves it via API, with no built-in front-end.
Headless CMS	A CMS that only manages/serves content via API; the developer builds the front-end that displays it.

Portable Text	Sanity's structured JSON format for rich text (instead of raw HTML), rendered into components on the front end.
React Three Fiber	A React renderer for Three.js, letting 3D scenes be built declaratively as React components.
Three.js	A JavaScript library for rendering 3D graphics in the browser using WebGL.
Spline	A visual, no-code tool for designing 3D scenes/models that can be exported and embedded in web apps.
GSAP	GreenSock Animation Platform — a JS library for high-performance, timeline-based UI animation.
Zustand	A minimal, hook-based React state-management library with far less boilerplate than Redux.
Redux Toolkit	The modern, opinionated standard for Redux; reduces boilerplate via createSlice and built-in immutable update handling (Immer).
Framer Motion	A React animation library for declarative transitions, gestures, and layout animations.
Resend	A developer-focused transactional email API for sending emails (confirmations, notifications) from app code.
Vercel	A cloud platform for deploying front-end apps (esp. Next.js), with Git-based deploys, preview URLs, and CDN hosting.
DNS	Domain Name System — translates human-readable domains into server IP addresses; configured to point a domain at a host.
Supabase	A backend-as-a-service on PostgreSQL, providing a hosted DB, auto-generated APIs, auth, and file storage.
PostgreSQL	An open-source relational (SQL) database, storing data in structured tables with defined relationships.
MongoDB	A NoSQL, document-based database storing flexible, JSON-like records instead of rigid tables.
SQL	Structured Query Language — the standard language for querying and manipulating relational databases.
Socket.io	A library enabling real-time, bidirectional communication between client and server, built over WebSockets (with fallback).
WebSocket	A network protocol providing a persistent, two-way connection between client and server, unlike request/response-only HTTP.
JWT (JSON Web Token)	A signed, self-contained token used to prove a user's identity across requests without server-side session storage.
bcrypt	A password-hashing algorithm that salts and one-way-hashes passwords before storage, so plaintext is never saved.

Cloudinary	A cloud service for storing, delivering (via CDN), and transforming (resizing/format) images and media.
Vite	A fast front-end build tool using native ES modules for near-instant dev-server startup and hot reload.
React Router	A library for client-side routing/navigation in single-page React applications.
Node.js	A JavaScript runtime (built on Chrome's V8 engine) that lets JavaScript run on a server, outside the browser.
Express.js	A minimal Node.js web framework providing routing and middleware for building APIs.
npm package	A reusable, versioned bundle of code published to the npm registry for others to install via `npm install`.
peerDependency	A dependency (e.g., React) a package requires the consuming app to provide, instead of bundling its own copy.
Tailwind CSS	A utility-first CSS framework providing small, composable classes (e.g., flex, px-4) applied directly in markup.
Figma	A collaborative interface-design tool used to create UI mockups/prototypes before or alongside development.
Git	A distributed version-control system tracking changes to code over time via commits and branches.
GitHub	A hosted platform for Git repositories, adding collaboration tools like pull requests, issues, and CI/CD.
CSV / Excel Export	Generating downloadable spreadsheet files (comma-separated values or .xlsx) from application data for reporting.
B2B	Business-to-business — a platform whose customers are other businesses rather than individual consumers.
CRUD	Create, Read, Update, Delete — the four basic operations performed on stored data.
Index (database)	A data structure on a column that lets a database find matching rows quickly instead of scanning every row.
Virtual DOM	React's in-memory copy of the real DOM, diffed against changes to compute the minimal real DOM update needed.

Final tip: Interviewers care most about whether you can explain your own decisions, not whether you recite definitions. For every "why did you use X" answer, be ready with the honest tradeoff — what you gained and what you gave up — since that's usually the real follow-up question.